

A Parallel-Pipelined Constant Geometry Algorithm (PCGA) for Computation of FFT on a Special Processor

Tze-Yun Sung

Electrical Engineering Department

Chung Hua University

Hisn-chu, Taiwan 300-12

Tel : 886-3-579-0902 Fax: 886-3-578-1150

E-mail : bobsung@btcx.com.tw

Abstract

In digital signal processing applications, we usually use DFT (discrete Fourier transform) to transfer signal from time-domain to frequency-domain. In past decades, there were many papers discussing and exploiting FFT (fast Fourier transform) architectures for hardware implementation.

In this paper, we proposed a novel VLSI FFT processor based on parallel-pipelined constant geometry algorithm (PCGA). This processor provides constant geometry for each stage of FFT, data reordering is not needed any more. Besides, the architecture of PCGA FFT processor is simple and regular, and the PCGA FFT processor always needs two-set of data RAM which acts as data storages for reading and writing data alternately.

Especially, the PCGA FFT processor increases throughput with increasing input points, and sufficiently exploits the advantage of parallel-pipelined computation for digital signal processing applications. Hence, comparing to conventional FFT processors, the PCGA FFT processor is more suitable and efficient for processing large number-point FFT. A novel implementation of the FFT processor should be

achieved.

Keywords: parallel-pipelined constant geometry algorithm (PCGA), FFT, CORDIC, VLSI.

1. Introduction

Discrete Fourier transform (DFT) is a very important tool for analyze digital signal running on digital computers [1]. The technique of fast Fourier transform (FFT) is widely applied on many application areas [2]; such as power spectrum estimation, filter simulation, radar systems, sonar systems, medical instruments, measurement instruments, and etc [3], [4], [5], [6], [7], [8], [9].

FFT is a fast computation algorithm proposed by Cooley and Tukey in 1965. FFT improves computation complexity from $O(N^2)$ to $O(N \log_2 N)$ and realizes DFT in real time applications of digital signal processing. It makes DFT algorithm perfect and practical.

In past decades, many papers proposed some FFT architectures and processors based on the fast algorithm proposed by Cooley and Tukey [10], [11], [12], [13], [14], [15], [16], [17]; nevertheless the

performance is not satisfied. Especially, when the number of input points is increased, the shuffle structure becomes more complex and operations of data reordering are also increased. Besides, the throughput and transformation time are also not satisfied, the system complexity $O(AT^2)$ becomes larger, when the number of input points is increased.

According to disadvantages of previous researches, we propose parallel-pipelined constant geometry FFT algorithm (PCGA). This algorithm improves computation complexity and hardware cost obviously.

2. Review of FFT Algorithms

• Constant Geometry FFT Algorithms

According to the fast algorithm proposed by Cooley and Tukey [10], $X(k) = DFT(x(n)) = \sum_{n=0}^{N-1} x(n)W_N^{nk}$, (1) where $k=0, 1, 2, \dots, N-1$.

Based on radix-2, from eq. (1), we obtain

$$\begin{aligned} X(k) &= \sum_{r=0}^{(N/2)-1} x(2r)W_N^{k(2r)} + \sum_{r=0}^{(N/2)-1} x(2r+1)W_N^{k(2r+1)} \\ &= \sum_{r=0}^{(N/2)-1} x(2r)W_{N/2}^{kr} + W_N^k \sum_{r=0}^{(N/2)-1} x(2r+1)W_{N/2}^{kr}, \end{aligned} \quad (2)$$

where $W_{N/2} = e^{-j\left(\frac{2\pi}{N/2}\right)}$.

$$\text{Let } A(k) = \sum_{r=0}^{(N/2)-1} x(2r)W_{N/2}^{kr}, \quad (3)$$

$$B(k) = \sum_{r=0}^{(N/2)-1} x(2r+1)W_{N/2}^{kr}, \quad (4)$$

where $k=0, 1, 2, \dots, N-1$.

$$X(k) \text{ is represented as } X(k) = A(k) + W_N^k B(k), \quad (5)$$

where $k=0, 1, 2, \dots, N-1$.

We may arrange sequence of input and output data

arbitrarily, and achieve the desired architecture; nevertheless we get used to input in normal order. Hence, we rearrange the flow graph of FFT to achieve the flow graph of constant geometry FFT computation with input in normal order which is shown in Fig. 2.1 [10].

In fact, the characteristic of flow graph of constant geometry FFT computation possesses constant geometry for each stage. The data of odd and even sequence are computed independently. Constant geometry structure reduces hardware complexity [18]. This characteristic is very suitable for parallel processing and VLSI implementation.

• Butterfly Computation

We observe eqs. (3) and (4), and Fig. 2.1, and find the basic computation. The basic operation is called “butterfly computation” which is shown in Fig. 2.2. In fact, the flow graphs of FFT are composed of many butterfly computations. The basic computation is represented as

$$X_{m+1}(p) = X_m(p) + W_N^r X_m(q), \quad (6)$$

$$X_{m+1}(q) = X_m(p) - W_N^r X_m(q), \quad (7)$$

where $0 \leq r \leq (N/2-1)$, $0 \leq p \leq (N/2-1)$, $(N/2-1) < q \leq (N-1)$, $q = p + N/2$, r , p and q are integers.

The butterfly computation can be performed by CORDIC arithmetic [18], which will be discussed in PCGA FFT processor.

3. PCGA FFT Processor

• Butterfly Processor for PCGA FFT Processor

Hardware implementation of CORDIC needs only shifters and adders, the simple hardware is very attractive. CORDIC is applied in signal processing widely today, we use CORDIC arithmetic to design butterfly processor.

In circular coordinate system of CORDIC (Coordinate Rotation Digital Computer), rotation mode can be represented as [18]

$$\begin{bmatrix} x_n \\ y_n \end{bmatrix} = K_c \begin{bmatrix} \cos z_0 & \sin z_0 \\ -\sin z_0 & \cos z_0 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}, \quad (8)$$

where $[x_0 \ y_0]$ is a input vector, z_0 is a rotation angle, K_c is a scale factor, and $[x_n \ y_n]$ is a output vector.

Because K_c is a constant, the scaling can be pre-processed or processed in parallel [19], [20]. The modified circular rotation can be embedded into butterfly computation as [19]

$$\begin{bmatrix} \text{Re}[X'_m(q)] \\ \text{Im}[X'_m(q)] \end{bmatrix} = \begin{bmatrix} \cos 2\pi/N & \sin 2\pi/N \\ -\sin 2\pi/N & \cos 2\pi/N \end{bmatrix} \begin{bmatrix} \text{Re}[X_m(q)] \\ \text{Im}[X_m(q)] \end{bmatrix}. \quad (9)$$

The butterfly computation is represented as

$$\begin{bmatrix} \text{Re}[X_{m+1}(p)] \\ \text{Im}[X_{m+1}(p)] \end{bmatrix} = \begin{bmatrix} \text{Re}[X_m(p)] \\ \text{Im}[X_m(p)] \end{bmatrix} + \begin{bmatrix} \text{Re}[X'_m(q)] \\ \text{Im}[X'_m(q)] \end{bmatrix}, \quad (10)$$

$$\begin{bmatrix} \text{Re}[X_{m+1}(q)] \\ \text{Im}[X_{m+1}(q)] \end{bmatrix} = \begin{bmatrix} \text{Re}[X_m(p)] \\ \text{Im}[X_m(p)] \end{bmatrix} - \begin{bmatrix} \text{Re}[X'_m(q)] \\ \text{Im}[X'_m(q)] \end{bmatrix}. \quad (11)$$

Fig 3.1 shows the modified CORDIC arithmetic unit and butterfly processor.

The butterfly processor [18], which is composed of a modified CORDIC arithmetic unit, two-adder and two-subtractor, performs function of butterfly computation and has multiplierless hardware. Hence, the butterfly processor has high performance and small area. These characteristics are very suitable for VLSI implementation [18], [19].

• Data RAMs for PCGA FFT Processor Using Memory Interleaving

The PCGA FFT processor possesses characteristics of parallel-pipelined processing and constant geometry for each stage. These characteristics can achieve fast transform and VLSI parallel-pipelined architecture. Due to Fig. 2.1, we find that the points of odd sequence and the points of even sequence are computed independently. According to the characteristics, we design a special structure of data RAM for FFT computation. The data RAM for 16-point PCGA FFT processor is shown in Fig. 3.2. The data RAM has input-data-buses and output-data-buses separately. In fact, twin data RAMs are read and written data alternately in the PCGA FFT processor. The data from one data RAM is sent to butterfly processors to process data, and the results are stored in the other data RAM. We use memory interleaving to arrange new data sequence.

The data RAM has four parts, and each part has dual data ports which separate input data port and output data port. “EVEN-HIGH” and “ODD-HIGH” of the data RAM are stored $X_m(q)$ ’s, and “EVEN-LOW” and “ODD-LOW” of the data RAM are stored $X_m(p)$ ’s.

According to number of butterfly processors, the data RAM has the same number of set of output data ports. In Fig. 3.2, one set of output data form “EVEN-HIGH” and “EVEN-LOW” of the data RAM, and the other set of data form “ODD-HIGH” and “ODD-LOW” of the data RAM. Two-set of data are sent to two butterfly processors simultaneously, and are computed in parallel.

• 16-Point PCGA FFT Processor

In the 16-point PCGA FFT processor, two butterfly processors compute data in parallel, and two-set of data RAM are read and written data alternately. The

16-point PCGA FFT processor and the timing diagram of reading and writing of butterfly processors are shown in Fig. 3.3.

In Fig. 3.3, we find that the data is read from *DATA RAM-I* with data sequence of 0, 8, 4, 12 and 1, 9, 5, 13, and the results are written into *DATA RAM-II* by data sequence of 0, 1, 8, 9, and 2, 3, during t_1 . Similarly, the data are read from *DATA RAM-I* by data sequence of 2, 10, 6, 14, and 3, 11, 7, 15, and the results are written into *DATA RAM-II* by data sequence of 4, 5, 12, 13, and 6, 7, during t_2 . Alternately, during t_3 and t_4 , the data are read from *DATA RAM-II* and written into *DATA RAM-I*.

According to above the sequence of reading and writing from data RAMs, we describe the timing diagram of 16-point PCGA FFT processor which is shown in Fig. 3.4. The PCGA FFT processor needs 8 time units to finish the computation of 16-point FFT.

• **Programmable 1024-Point PCGA FFT Processor and Multiprocessor System**

The programmable 1024-point PCGA FFT processor is designed and evaluated on Xilinx XC2V3000 FPGA. This processor can process 1024-point FFT, and includes 2K words data RAM, 0.5K words twiddle-factor ROM for butterfly processors, ten butterfly processors, digital serial peripheral interfaces (DSPIs) for interfacing to digital signal source and master processor, address decoders for memory interleaving, and controller for managing system operation. The master processor can input commands to the programmable PCGA FFT processor by number of point ($2^n \leq 1024$), and a “single/ pipelined” control. Some programmable PCGA FFT processors can be cascaded to perform a multiprocessor system controlled by “single/pipelined”. The multiprocessor system has a

parallel-pipelined processing architecture to increase system throughput. The multiprocessor system for 16-point FFT computation is shown in Fig. 3.5. This multiprocessor system can process 16-point FFT computation in parallel and pipeline. The latency time is eight time units, but throughput is improved to produce 16-point data for each two time units.

4. Discussion and Conclusion

The PCGA FFT processor always needs two sets of data RAM which acts as data storages for reading and writing data alternately. This architecture provides constant geometry for each stage of FFT, data reordering for each stage is not needed any more. So, the hardware implementation of PCGA FFT processor is simpler than previous designs [12], [13], [14], [15], [17], and the performance of PCGA FFT processor is better than many previous designs [12], [14], [17].

The hardware code is written in Verilog running on SUN Blade 1000 workstation under the Modelsim simulator and emulated on Xilinx XC2V3000 FPGA [21]. It is designed to evaluate the architecture and provide an embedded core for digital signal processing applications [22]. Hence, we have a table related to number of points and butterfly processors, memory size and processing time. This table is shown in Fig. 4.1.

We also analyze and simulate a pipeline FFT processor which was proposed by S. I. Sayegh [17]. Sayegh proposed a pipeline processor for mixed-size FFT's, the design of FFT processor is very practical and suitable for VLSI implementation. But Sayegh's architecture needs more complex data switching for each stage, more butterfly processors and more computation time. The table related to butterfly

processors, memories and computation time is also shown in Table 4.1.

The comparison of computation time is shown in Fig. 4.1. Besides, we also analyze and write an AT^2-N diagram which is shown in Fig. 4.2, and find that the PCGA FFT processor improves the disadvantage of Sayegh's FFT processor obviously.

Table 4.2 compares the hardware requirement for different FFT processors. A parallel complex multiplier needs four multipliers at least, but a CORDIC arithmetic unit performs complex multiplication and needs no multiplier. The PCGA FFT processor achieves a significant savings of memory [23], [24], and requires less number of arithmetic units and hardware [23], [24], [25].

Table 4.3 shows that the comparison of hardware utilization and computational efficiency. The PCGA FFT processor is higher than the implementations of Hui's [25], Chang's [24] and Bi's [13].

Especially, the PCGA FFT processor increases throughput with increasing input points, and sufficiently exploits its advantage of parallel-pipelined computation for digital signal processing applications. Hence, comparing to the conventional pipelined FFT processor architecture [12], [13], [14], [15], [17], [23], [24], [25], the PCGA FFT processor is more suitable and efficient for processing large number-point FFT computation, and achieves high performance and small area. In past decade, the research of computation error and accuracy analysis of FFT is rarely; we analyze the computation error of core arithmetic and propose the model of fixed point computation error. The design reference for the PCGA FFT processor is also proposed. The numerical accuracy of the PCGA FFT processor has been analyzed, and the computation error would be predicted. The hardware would also be saved. A novel implementation of the FFT processor should be achieved.

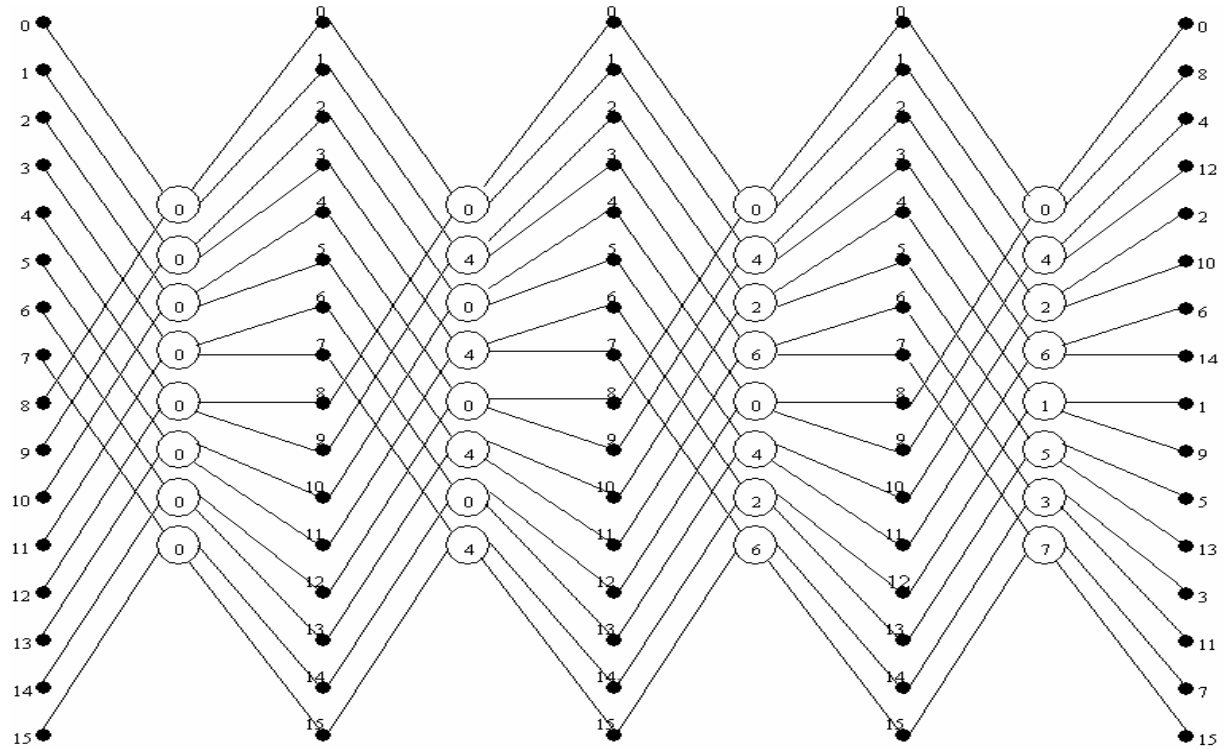


Fig. 2.1 The flow graph of constant geometry FFT computation with input in normal order.

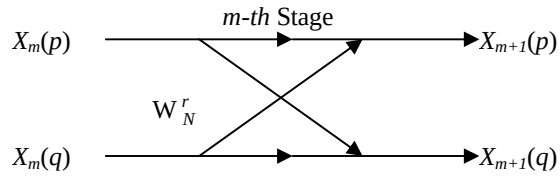
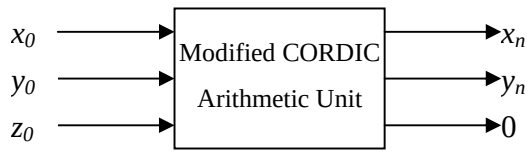
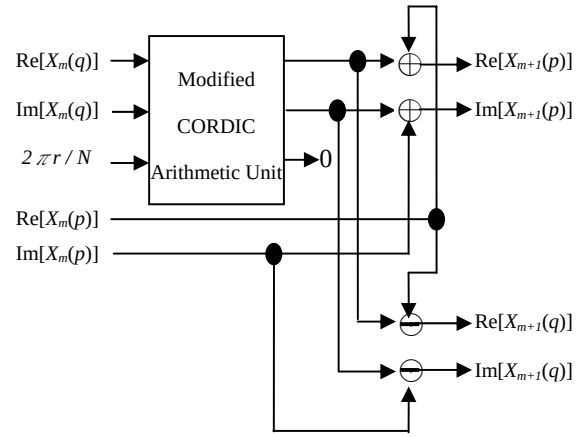


Fig. 2.2 The flow graph of butterfly computation



(a) The modified CORDIC arithmetic unit



(b) The butterfly processor

Fig. 3.1 The modified CORDIC arithmetic unit and the butterfly processor

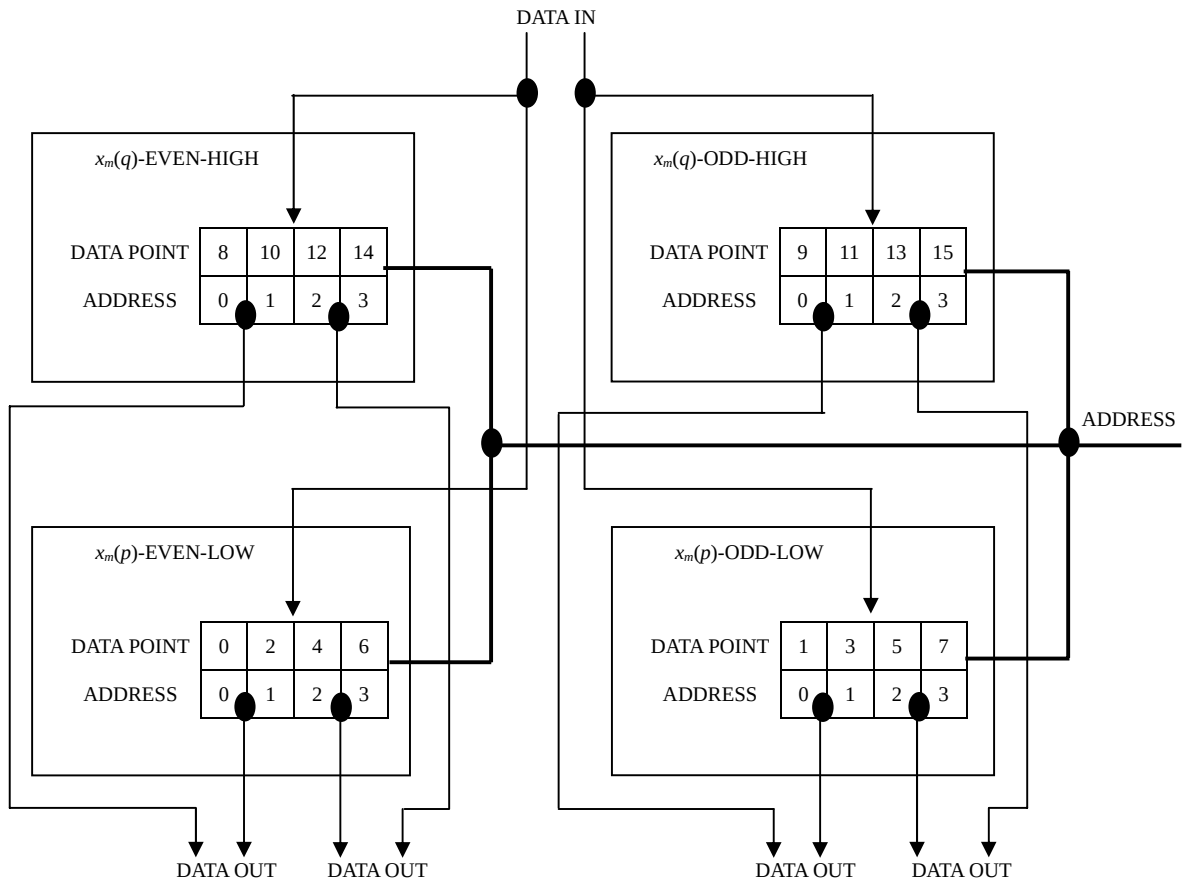
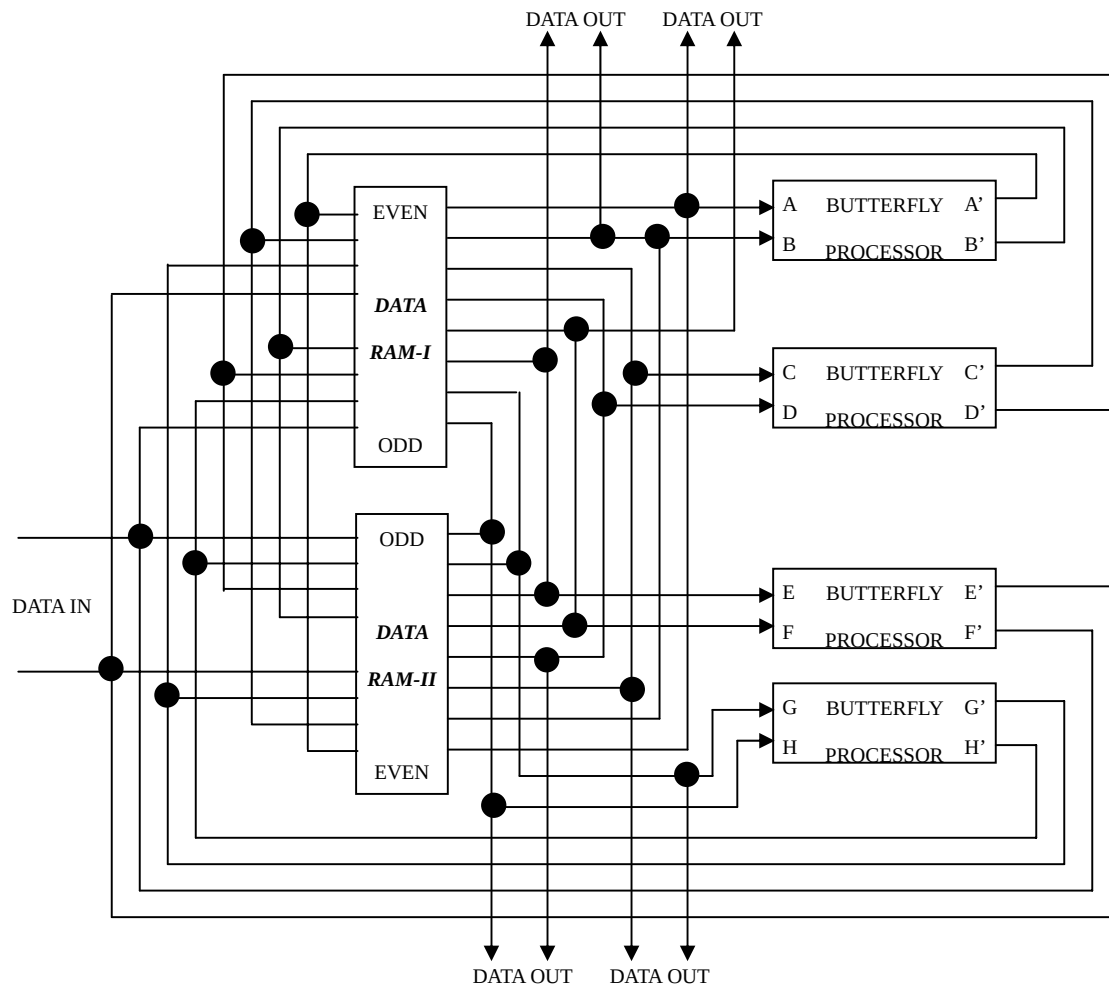


Fig. 3.2 The data RAM for the 16-point PCGA FFT processor



(a) The 16-point PCGA FFT processor

Reading Mode:

Timing Sequence	A	B	C	D	E	F	G	H
t_1	0	8	4	12	1	9	5	13
t_2	2	10	6	14	3	11	7	15

Writing Mode:

Timing Sequence	A'	B'	C'	D'	E'	F'	G'	H'
t_1	0	1	8	9	2	3	10	11
t_2	4	5	12	13	6	7	14	15

(b) The timing diagram of reading and writing butterfly processors

Fig. 3.3 The 16-point PCGA FFT processor and the timing diagram of reading and writing of butterfly processors

t	Data RAM- I	Data RAM- II	Output
t_1	10 14 11 15 2 6 3 7	8 10 9 11 0 2 1 3	----
t_2	----	8 10 12 14 9 11 13 15 0 2 4 6 1 3 5 7	----
t_3	8 10 9 11 0 2 1 3	10 14 11 15 2 6 3 7	----
t_4	8 10 12 14 9 11 13 15 0 2 4 6 1 3 5 7	----	----

t_5	10 14 11 15 2 6 3 7	8 10 9 11 0 2 1 3	----
t_6	----	8 10 12 14 9 11 13 15 0 2 4 6 1 3 5 7	----
t_7	----	10 14 11 15 2 6 3 7	$X_0 X_8$ $X_1 X_9$ $X_4 X_{12}$ $X_5 X_{13}$
t_8	----	-----	$X_2 X_{10}$ $X_3 X_{11}$ $X_6 X_{14}$ $X_7 X_{15}$

Fig. 3.4 The timing diagram of 16-point PCGA FFT processor

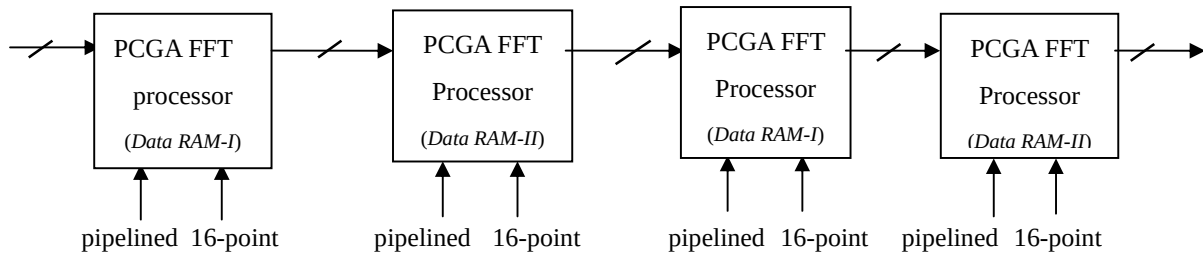


Fig. 3.5 The multiprocessor system for 16-point FFT computation

No. of Point (N)	No. of Butterfly Processor (A)		Memory Size (Words)		Computation Time (T)	
	PCGA	Sayegh[17]	PCGA	Sayegh[17]	PCGA	Sayegh[17]
8	2	3	16	12	6	10
16	4	4	32	28	8	22
32	4	5	64	60	20	46
64	6	6	128	124	36	94
128	6	7	256	252	76	190
256	8	8	512	508	128	382
512	8	9	1024	1020	288	766
1024	10	10	2048	2044	520	1534

Table 4.1 Comparison of PCGA FFT Processor and Sayegh's processor

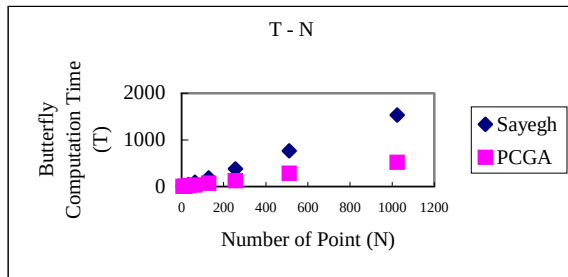


Fig. 4.1 The comparison of computation time

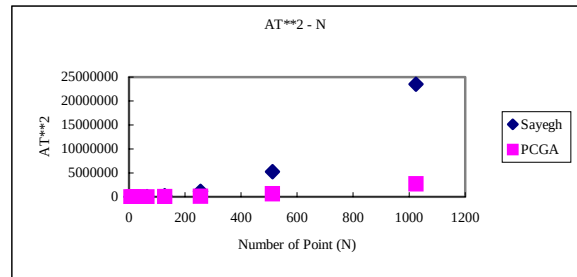


Fig. 4.2 The comparison of system complexities

No. of Points	No. of CORDIC*	No. of Complex Multipliers			Data RAM Size			
	PCGA	Chang[24]	Hui[25]	Swatzlander[23]	PCGA	Chang[24]	Hui[25]	Swatzlander[23]
16	4	3	3	3	32	20	48	48
64	6	6	6	6	128	76	160	160
256	8	9	9	9	512	302	640	640
1024	10	12	12	12	2048	1210	2560	2560

No. of Points	Twiddle-factor ROM Size			
	PCGA	Chang[24]	Hui[25]	Swatzlander[23]
16	8	8	16	16
64	32	32	64	64
256	128	128	256	256
1024	512	512	1024	1024

Table 4.2 The comparison of FFT processors (*CORDIC arithmetic unit: multiplierless)

Computational Efficiency of Arithmetic	PCGA	Bi[13]	Hui[25]	Chang[24]
Addition	100%	50%	100%	~100%
Multiplication	-	75%	100%	100%
CORDIC*	100%	-	-	-

Note: *CORDIC arithmetic operation for PCGA.

Table 4.3 The comparison of hardware utilization and computational efficiency

References

- [1] W. T. Cochran, "What is the Fast Fourier Transform?" *IEEE Trans. on Audio and Electroacoustics*, Vol. AU-15, pp. 45-55, June 1967.
- [2] J. W. Cooley, P. Lewis, P. Welch, "Application of the Fast Fourier Transform to Computation of Fourier Integrals, Fourier Series, and Convolution Integrals," *IEEE Trans. on Audio and Electroacoustics*, Vol. AU-15, pp.79-84, June 1967.
- [3] J. F. Kaiser, "The Digital Filter and Speech Communications," *IEEE Trans. Audio Electroacoustics*, Vol. AU-16, pp.180-183, June 1968.
- [4] H. C. Andrews, K. L. Caspari, "A Generalized Technique for Spectral Analysis," *IEEE Trans. on Comput.*, Vol. C-19, pp.16-25, Jan. 1970.
- [5] S. M. Kay, S. L. Marple, "Spectrum analysis-A modern perspective," *Proc. IEEE*, Vol. 69, pp. 1380-1419, Nov. 1981.
- [6] E. A. Robison, "The Special Issue on Spectral Estimation," *Proc. IEEE*, Vol. 70, pp. 883-1125, Sept. 1982.
- [7] M. N. S. Swamy, K. S. Thyagarajar, "Frequency Transformation of Digital Filters," *Proc. IEEE*, Vol. 65, pp. 165, Jan. 1977.
- [8] A. G. Constantinides, "Spectral Transformation for Digital Filters," *Proc. IEEE*, Vol. 117, pp. 1585-1590, Aug. 1970.
- [9] Alan V. Oppenheim, Ronald W. Schaffer, *Discrete-time Signal Processing*, Second Edition, Prentice-Hall, Inc. Chapter 9, pp.629-692, 1999.
- [10] J. W. Cooley, J. W. Tukey, "An Algorithm for the

- Machine Calculation of Complex Fourier Series,” *Math. Comput.*, Vol. 19, pp. 297-301, 1965.
- [11] H. L. Groginsky, G. A. Works, “A Pipeline Fast Fourier Transform,” *IEEE Trans. on Comput.*, Vol. C-19, No. 11, pp.1015-1019, Nov. 1970.
- [12] E. H. Wold, A. M. Despain, “Pipeline and Parallel-pipeline FFT Processor for VLSI Implementations,” *IEEE Trans. on Comput.*, Vol. C-33, No. 5, pp.414-426, May 1984.
- [13] G. Bi, E. V. Jones, “A Pipelined FFT Processor for Word-sequential Data,” *IEEE Trans. on Acoustics, Speech, and Signal Processing*, Vol. 37, No. 12, Dec. 1989.
- [14] B. Gold, T. Bially, “Parallelism in Fast Fourier Transform Hardware,” *IEEE Trans. on Audio and Electroacoustics*, Vol. AU-21, pp. 5-16, Feb. 1973.
- [15] J. M. Glass, “An Efficient Method for Improving Reliability of a Pipeline FFT,” *IEEE Trans. on Comput.*, Vol. C-29, No. 11, pp. 1017-1020, Nov. 1980.
- [16] A. M. Despain, “Very Fast Fourier transform Algorithms Hardware for Implementation,” *IEEE Trans. on Comput.*, Vol. C-28, pp. 333-341, May 1979.
- [17] Soheil I. Sayegh, “A Pipeline Processor for Mixed-size FFT’s,” *IEEE Trans. on Signal Processing*, pp. 1892-1990, Aug. 1992.
- [18] T. Y. Sung, Y. H. Hu, H. J. Yu, “Doubly Pipelined CORDIC Array for Digital Signal Processing Algorithms,” *Proc. IEEE Int. Conf. Acoustics, Speech, and Signal Processing*, Vol. 2, pp.1169-1172, Apr. 1986.
- [19] T. Y. Sung, Y. H. Sung, “Smart Design of CORDIC Processor for Signal Processing,” *Chung Hua Journal of Science and Engineering, Special Issue on Electrical and Computer Engineering*, School of Engineering, Chung Hua University, Hsin-chu, Taiwan, Vol.2, 2004.
- [20] T. Y. Sung, Y. H. Sung, “Numerical Accuracy and Hardware Trade-offs for CORDIC Arithmetic for DSP Applications,” *Chung Hua Journal of Science and Engineering*, School of Engineering, Chung Hua University, Hsin-chu, Taiwan, Vol.1, 2004.
- [21] T. Y. Sung, C. S. Chen, “A Parallel-Pipeline FFT Processor Using Memory Interleaving,” *Chung Hua Journal of Science and Engineering*, School of Engineering, Chung Hua University, Hsin-chu, Taiwan, Vol.1, 2004.
- [22] T. Y. Sung, Y. H. Sung, “SoC Implementation of A Programmable High-Performance FFT Processor,” *Technical Report (BTCX-SoC-031133)*, BTCX Electronic Corp., Science Based Industrial Park, Hsin-chu, Taiwan, Dec. 2003.
- [23] E. E. Swartzlander, W. K. W. Young, S. J. Joseph, “A radix-4 delay Commutator for Fast Fourier Transform Processor Implementation,” *IEEE J. Solid-State Circuits*, Vol. SC-19, pp.702-709, Oct. 1984.
- [24] Y. N. Chang, K. K. Parhi, “An Efficient Pipelined FFT Architecture,” *IEEE Transactions on Circuits and Systems-II: Analog and Digital Signal Processing*, Vol. 50, No.6, pp.332-325, June 2003.
- [25] C. C. W. Hui, T. J. Ding, J. V. McCanny, “A 64-point Fourier Transform Chip for Video Motion Compensation Using Phase Correction,” *IEEE Journal of Solid-State Circuits*, Vol.31, pp. 1751-1761, Nov. 1996.